Architecture of an Interoperable IoT Platform Based on Microservices

Tomislav Vresk* and Igor Čavrak**
*Končar elektronika i informatika, Zagreb, Croatia
**University of Zagreb, Faculty of Electrical Engineering and Computing, Zagreb, Croatia
tvresk@koncar-inem.hr, igor.cavrak@fer.hr

Abstract - The vision of the Internet of Things enabled the development of a wide spectrum of services, applications and ecosystems, deemed infeasible not long ago. However, lack of standardization poses a number of questions still requiring proper addressing. Due to the need of supporting large number of users and significant data processing throughput, Internet of Things requires a specific approach towards the problem of providing sufficient scalability and performance, clearly pointing towards the distribution of effort among a large number of small and specialized services. Reflecting on the importance of coexistence of heterogeneous systems supporting the Internet of Things, this paper presents an overview of the specific problems inherent to the IoT and the proposal of an architecture of a microservice based middleware aimed at connecting heterogeneous IoT devices. The middleware functionality is achieved irrespective of the size and complexity of a given device network, both from the data model aspect and from the aspect of connecting existing and newly created middleware components.

I. INTRODUCTION

The emerging communication technology breakthrough, based on the concept of each device having the ability to communicate with its surrounding environment while being able to take leverage of data gathered by other nearby connected devices, has grown into a phenomenon called the Internet of Things (IoT). Studies conducted in the business sector indicate that currently 20% of American households have already integrated some kind of IoT platform, while 68% of them intend to do it within the next 10 years. Independent market research and advisory firms forecast 6.4 billion connected things will be in use worldwide in 2016, up 30% from 2015, and will reach 20.8 billion by 2020 [1].

Although the concept of a global connectivity represents the foundation of the IoT that jump started the fourth industrial revolution, due to complexity inherited trough ability to integrate infinite number of technologies and multi-purpose communication protocols it also reveals the problems which lie in the nature of the IoT itself.

The paradigm raises questions on not only the security and privacy aspects but calls into question the laws governing the administration of such a network [2]. For the IoT to reach its full potential, a single internationally accepted communications standard is needed [3]. Even though standardization will have a significant impact on the paradigm evolution, the possibility of enforcing *de jure* and/or *de facto* standards by the predominant companies should not be ignored; monopoly on the interoperability technology may have rights associated with it, owned by one or several market players. In addition, such rights may be exploited to generate monopoly rents, counteracting the competition in standard-based interoperable products and services.

Properly implemented standard guarantees interoperability, which represents a crucial starting point for the successful integration and information sharing between distributed systems. Energy efficiency and information security are just some of the industry segments where standardization represents a mandatory obligation. In order to overcome this critical barrier, it is important to enable global ubiquitous connectivity through usage of publicly available communication standards, so that the data generated by devices can be shared and cross pollinated to create new and useful cross-functionality. At the same time, there are other segments and areas that do not require any way of standardization. Therefore, it is perfectly legitimate to ask whether is it really possible to standardize IoT, and more importantly, is standardization the only prerequisite standing in the way of enabling the global interoperability.

To address the scalability and interoperability problems, we define an IoT concept and platform based on orchestration of different IoT system components (devices, data sources, data processors, storage etc.) implemented in the form of microservices. The microservice based approach aims to hide the complexity stemming from variation of end-device properties by employing a uniform approach towards modelling of both physical and logical IoT devices and services, including uniform device addressing schemas. The approach also fosters interoperability and extensibility by encapsulating diverse communication protocols into proxy microservice components thus allowing for easy replacement of existing and adding new system components, laying the groundwork for open and dynamic IoT ecosystems.

Section II of this paper addresses the inherent problem of scalability in IoT systems and the microservice-based approach to tackling it. Section III describes our proposal of IoT-specific microservice architecture and organization. Section IV describes the example implementation of an

IoT system based on the microservice approach, while Section V concludes the paper.

II. MICROSERVICES

The rapid development of technology enabled ubiquitous connectivity resulting in a continuous growth of a number of connected devices, but it also revealed many challenges related to development and architecture of current information systems. While the network traffic generated by smart phones depends solely on the number of active users, IoT solicits a different approach where network data traffic results from communication among a vast number of both permanently and intermittently connected devices. Due to cohesion of wide variety of solutions and interacting devices, it is almost impossible to accurately and effectively predict data traffic patterns. As a result, such erratic behaviour can lead to exponential growth of network traffic and unpredictable system load.

In order to handle the growing amount of work, such as demand spikes, it is essential to implement scalability mechanisms and strategies that will maintain the system stability. There are two basic strategies for scaling: vertical and horizontal [4]. In case of vertical scaling, additional resources are added to a single processing node. As a result, the node can handle more work and provide additional capacities; however, vertical scaling is obviously limited by factors such as cost effectiveness, physical constraints and availability of specialized hardware. In contrast, horizontal scaling adds more nodes to the overall system. Opposite to vertical scaling, horizontal scaling represents a set of design practices and planned activities leading to an inherent distribution within the system. If the system supports horizontal scaling, it can be theoretically enlarged to thousands of machines. This unique ability defines the horizontal scaling approach as the key building stone of a large-scale architecture design.

The microservices approach, a relatively new term in the software architecture patterns [5], could present one of the solutions to the scalability problem in the IoT domain. Microservice pattern advocates organizing distributed applications as a suite of services, where each of the services is running in its own independent process context designed to segregate one service from another. Internal cross-service communication is implemented trough proprietary interfaces, their implementations predominantly relying on lightweight TCP/UDP-based application protocols.

At the core of the microservices approach lays the possibility to independently deploy and scale each service; a service may be deployed as several instances and different services may be hosted on the same server. When building and deploying distributed systems, the focus on automated provisioning is crucial. Such tools and services, essential for establishing the service provisioning and management chain, can be built using any programming language or development framework.

Each microservice is responsible for one or more closely related functions. There are a couple of different ways of decomposing the application into services. One approach is to use verb-based decomposition and define services that implement a single use case. The other option is to decompose the application by noun and create services responsible for all operations related to a particular entity such as customer management. An application might use a combination of verb-based and noun-based decomposition.

Opposite to the microservice architecture, the modern monolithic architecture provides a natural way of development, while combining all logic into a single process divided and organized into classes, methods and packages [6]. Such way allows the use of a deployment pipeline to ensure changes are properly tested and deployed into the processing environment. At the same time, a small change in the application code requires the application/service to be completely rebuilt and redeployed.

Taking into consideration that IoT relies on the integration of a large number of technologies, it is reasonable to conclude that a monolithic based approach does not represent a suitable solution. Advances in technology have driven a large amount of new architectures and design patterns. As such, the feature-based framework microservice approach allows the reuse of components leading towards the highly scalable service-oriented ecosystem.

III. IOT MICROSERVICE COMPONENTS

Currently, the main focus of the ongoing IoT-related research is on resolving the connectivity challenge; however, there is something that connects them all: telemetry. Even though telemetry represents one of the crucial functionalities of each connected device, there still remain more communication patterns, such as commands, queries and notifications, and accompanying asynchronous and synchronous data processing, that are often completely excluded from taking into consideration.

Synchronous or transactional operations in an inherently asynchronous IoT environment tend to introduce additional layers of complexity by employing distributed transaction management mechanisms to tie various actions together. As such, implementation of all communication towards other distributed components trough a single interoperable interface represents one of the biggest challenges in the considered architectural design.

From a high level perspective, the microservice component model can be interpreted as a black box decomposed into logical groupings - layers (Fig. 1). Layers help to differentiate between different kinds of tasks performed by components, making it easier to create a design that supports reusability of components. The basic function set includes cross-service API responsible for service data context exchange, coordination and remote service management.

End user authentication is implemented using token-based authentication. It allows consumers such as end users or services to provide credentials in order to obtain a token which allows fetching a specific resource. Once the

token has been obtained, the owner is granted access to a specific resource for a specific time period.

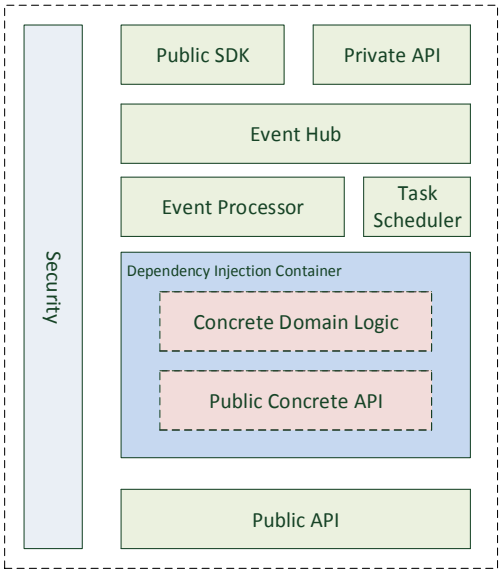

Figure 1. Microservice high level overview

Interactions among different microservices are restricted to data exchange using only microservice-exposed interfaces. By utilizing distributed data flows, services can make use of computing resources across the network. Note that such robust data flow management is required to handle the velocity, variety, and volume of exchanged data. The implementation of process logic is facilitated by inheriting functionality from pre-existing microservice core components, with the addition of communication mechanism implementations by using framework-provided API-s. By taking such an approach, a concrete microservice instance can represent a data management REST interface [7], CoAP end point [8] or simply a permanent storage data historian.

Rapidly changing technology represents one of the biggest challenges in achieving ubiquitous connectivity; every day, new technology is developed while old technologies being obsoleted or upgraded. Currently, there is a broad set of protocols being promoted as the silver bullet of IoT communication for the higher-level M2M protocol in the protocol stack [9], bringing the standard IP-centric protocols into the realm of smart devices and smart objects. One of the biggest challenges that IoT ecosystems face are the presence of low-powered devices; such devices need to function for prolonged periods of time in unfriendly environments, restrained by limited power resources and supporting frequent data exchanges over lossy communication links. These unique challenges make the as-is use of the existing TCP/IP-based communication protocols less than ideal.

Several factors pose significant impact on device battery life; the capacity being the main one. However, in a battery life estimation equation, capacity is a constant, while consumption appears to be only other relevant factor. Similar to battery life estimation, there are many factors that determine power consumption; frequency, payload size and communication protocol are just the essential ones. One of the key energy/power considerations in wireless communications for low-powered devices is the duty-cycle of the wireless radio transceiver. Communication protocols that allow the transceiver to remain inactive for longer periods of time are far more energy efficient than those requiring the radio to be almost constantly powered on. By taking all variables into consideration, it is reasonable to conclude that such unique scenario requires support for lightweight communication mechanism. As such, RESTful HTTP over TCP is particularly attractive for connecting consumer premise devices, given the near universal availability of HTTP stacks for various platforms.

A. Data Model

Data are at the heart of every information system. Without connected devices being able to capture, transfer, analyse, report and act on data, the benefits of the IoT would not be achievable. As as result, we are challenged with questions: who owns this data and who should have access to it? Issues related to interoperability, automation, and data analytics naturally lead to semantic-oriented perspective [10]. Applying semantic technology promotes interoperability among resources, information models, data providers and consumers, and facilitates data access, integration and knowledge extraction.

Prior to describing our proposed data model, key concepts such as *virtual device*, *thing* and *resource* will be introduced (Fig. 2). A *thing* can be any object, even abstract data context in the real world. *Things* are represented as *logical devices* having a unique ID, name, type, attributes and a list of *logical nodes*. A *logical device* contains one or more *logical nodes*, which represent various components of the physical device. A *resource* is a computational element providing access to a particular *logical device element*. In order to represent a physical device as a logical device, decoupling into corresponding virtual components in most cases will not provide enough relevant data. Attributes are part of the information model used for resource description, and can include additional context data like resource location, sensor accuracy etc.

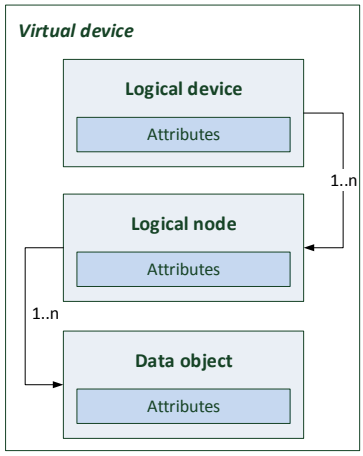

Figure 2. Logical device data model hierarchy

As an example, the modelled IoT device could contain a thermal energy smart meter logical device extended with surrounding environment temperature and a day ahead thermal energy usage estimation. The logical device would consist of two logical nodes: *measurement* and *estimation*. *Measurement* logical node would contain surrounding environment properties such as environment temperature data object and thermal energy smart meter data set. *Estimation* logical node would contain a day ahead thermal energy usage estimation data object representing a data output of analysis performed by concrete microservice via data inputs gathered from other microservice data outputs.

The biggest challenge arising from digitalization of physical assets is preserving the sufficient level of detail. Data can be defined as a representation of facts, concepts or instructions in a formalized manner suitable for communication, interpretation, or processing by human or electronic machine. When data are processed, interpreted, organized, structured or presented so as to make them meaningful or useful, they are called information. Such information provides context for data and therefore can be represented with a set of abstract primitive data types that are used to define data object in a virtual conceptual model.

B. Address Model

The primary objective of the address space is to define a method for data providers to represent resources to potential users. The object data model has been designed to meet this objective, with resources being addressable using the uniform addressing scheme. Address space can be directly mapped to REST interface exposing resources in a RESTful service as unique URL-s. However, logical device data elements are subject to structural changes, thus it is much more difficult to implement human readable addressing scheme compared to other more conventional approaches. Depending on the usage context, we are proposing two alternative addressing schemes for accessing the desired resource; *unified human readable scheme* and *unified global scheme*.

The *unified human readable scheme* aims to organize resources using the meaningful human readable URLs. As previously defined, each logical device represents a virtual model of some physical object and as such can be differentiated to less complex components. Because of the tree-like properties of the model, this path resembles a fully-qualified file-name notation. The path lists all the objects on the route from the root of the model to the object in question. Furthermore, fully-qualified file-name notation usually has a fixed delimiter between object names, and unified human readable notation uses slash to separate the logical device from the rest of the path. As an example of the human readable scheme, the setup contains a weather station model composed of multiple logical nodes containing the latest temperature information:

uhm/ws-1-location/ measurement -latest-temperature

uhm: address model

ws-1-location: logical device

measurement: logical node

latest: logical node containing leaf data objects

temperature: data object

To satisfy the REST constraints, each URL should map to a single resource and all access to this resource should be provided using that specific URL. However, opposite to other addressing concepts, this notation uses paths that resemble a fully-qualified file-name notation, meaning that they can easily be mapped to a URL.

In the *unified global scheme*, each resource is automatically assigned a globally unique resource identifier. With reference to a given set of objects, the unique identifier is any identifier which is guaranteed to be unique among all identifiers used for those objects and for a specific purpose within the distributed system. Although the unified global scheme is much easier to implement and, unlike human readable scheme, is in full compliance with RESTful nature, in the end it represents a trade off between good usage practices and human readable semantics.

As an example of the unified global addressing scheme, the setup contains a weather station model composed of multiple logical nodes containing the latest temperature information:

ugm/data-object/5428d753-b033-4c4e-98f9-d9c1b4a1f8ff

uum: address model

data-object: resource type

IV. IMPLEMENTATION SCENARIO

The typical cloud computing architecture, presented on Fig. 3, introduces architecture that meets all the relevant IoT criteria. The purpose of this example scenario is to create cohesion between devices using standard communication protocol and third party web service which involves establishment of microservice data flow chain whose outputs act as inputs for predictive analytics processing, resulting with the future thermal energy consumption estimation.

The scenario includes a number of smart meters installed on a chosen location. Since the energy efficiency segment is subject to standard, the status of individual meters is read via an open communication protocol. Most of the energy meters are installed in a constant power supply capable environment, but due to specific environmental properties (i.e. prohibitive costs) they tend to have a limited IP-capable connectivity. Even though this problem has been partially minimized with the

advancement of technology trough GPRS/3G and upcoming LTE-M/NB-IOT [11], Wireless M-Bus based solutions [12] are still having primary role in meter reading system architectures. Such Wireless M-bus based energy meters represent non IP capable devices and require additional IP-capability integration. In order to provide the IP-capability, Wireless M-Bus to Modbus TCP converter is integrated into existing network.

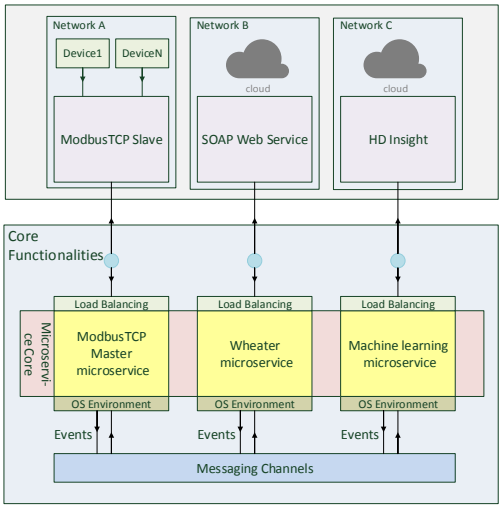

Figure 3. High level platform overview

The architecture consists of several layers. The first layer contains the load balancing nodes. The following layer contains microservice applications responsible for transfer, processing and transformation of data trough a dataflow pipeline. The pipeline encapsulates an entire series of computations that accepts input data from both internal and external sources, transforms that data to provide useful intelligence, and produces output data. Opposite to the traditional generic pipeline approach, microservice based dataflow pipeline supports bi-directional communication through usage of integrated mechanisms. For the purposes of current architecture, a dataflow pipeline is formed by integrating three microservices: Modbus TCP master, meteorological (weather) service and machine learning predictive analytics.

Even though current architecture is based on broad research and predefined functional requirements, some real world problems were discovered only during the implementation phase. Such problems could be classified into two main groups: problems originated due to lack of chosen development framework knowledge and problems caused due to communication protocol limitations.

A. *Modbus TCP master microservice*

Modbus is considered an application layer messaging protocol, providing master/slave communication between devices connected through buses or networks [13]. Since Modbus protocol is just a messaging structure, it is independent of the underlying physical layer, and is therefore suitable for TCP encapsulation [14]. As such, Modbus TCP uses the client/server architectural style.

Opposite to modern event-driven publish/subscribe communication protocols, Modbus TCP is based on passive dataflow mechanisms requiring specific time interval polling. Designed and built around base microservice core, the developed Modbus TCP microservice provides essential Modbus TCP protocol stack support for data acquisition of Wireless M-Bus over Modbus TCP gateway connected devices. Originally published in the 80s, simple and robust, during the past years it has become de facto standard communication protocol. However, the address field in the Modbus-defined frame is only 8 bits wide. As such, Modbus is restricted to addressing 247 devices on one data link, limiting the number of field devices that may be connected to a master station (Fig. 4). To get beyond this limit, a protocol modification could be made to use two-byte address format, requiring both master and slaves to support it. However, such an approach would not be in compliance with the standard and would not represent a viable solution.

Relying on the key concepts of this paper, a solution to this problem could be found by adding multiple instances of the same microservice with unique IP or port parameters, distributing the global data context over multiple nodes without the need for complex protocol stack modifications. Defined by the standard, each Modbus capable device bases its data model on a set of tables with distinguishing characteristics.

Besides the implementation of Modbus protocol stack, Modbus TCP microservice includes additional key functionalities including task based slave context polling, in-service data historization and cross-service data exchange.

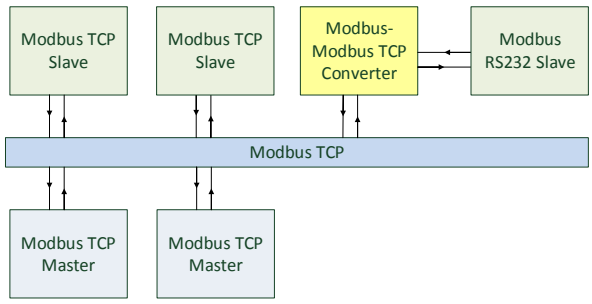

Figure 4. Modbus TCP network architecture

B. *Meteorological microservice*

In residential IoT systems (home automation scenarios) environmental temperature represents one of the key points in the energy consumption estimation equation. By examining the present energy consumption estimation model, it is clear that thermal energy consumption in relation with time does not represent a complete set of parameters required for performing the future estimation analysis successfully.

The main role of meteorological microservice is microclimate data acquisition, in-service data historization and cross-service exchange. Opposite to Modbus TCP microservice, which sets focus on implementation of Modbus communication protocol stack, meteorological microservice encapsulates non-standard web service in order to implement microclimate data acquisition for a preconfigured geographical location.

C. Machine learning microservice

Machine learning is quietly taking over in the mainstream and in a short period of time it has become a vital segment of every intelligent application [15]. The main role of machine learning predictive analytics microservice is data acquisition, model processing and model output stream warehousing.

Currently there is no standard methodology by which to carry out data mining projects. Due to lack of standardization each manufacturer uses its own in-house designed predictive model development methodology. However, our main focus is set on integration of Microsoft HD Insight (Hadoop) [16] framework based on CRISP-DM methodology [17]. The CRISP-DM process model aims to make large data mining projects less costly, more reliable, repeatable, manageable and faster. The process consists of the following four steps.

Defining goals - defining the right problems and goals represents the first step in predictive analytics. This step starts by asking what do we want to achieve and what is the final result.

Obtaining the data - after the analytical environment has been set up and the goals defined, the next step includes importing the right dataset upon which data model will be created. Even though data can be imported from many sources in this particular scenario processing dataset includes data gathered from both Modbus TCP master and meteorological microservices. The resulting data set contains current consumption, ambient temperature, corresponding data point time stamps and assigned geographical locations.

Data pre-processing - a dataset usually requires a certain amount of pre-processing before it can be analysed. Pre-processing available data involves adjusting the available data using the standard API which allows input row data manipulation and transformation using available methods such as merging datasets, grouping and summarizing data, checking for missing values and replacing them with appropriate values and converting values to another type.

Selecting and applying the learning algorithm - predictive analytics is the process of building models from historical or current data in order to forecast future outcomes. Predictive modelling is a process used in predictive analytics to create a statistical model of future behaviour.

A predictive model is made up of a number of predictors, variable factors that are likely to influence future behaviour or results. Data is collected for the relevant predictors, a statistical model is formulated, predictions are made and the model is validated (or revised) as additional data becomes available.

In order to estimate the future energy consumption simplest linear regression model has been chosen for our example system [18]. Regression analysis is a data mining process that identifies a model that depicts the correlation between an outcome and one or more predictors. The result is a trained regression model that can be used to score new samples to make predictions.

V. CONCLUSION

The vision of the 4th industrial revolution, triggered by the innovative technological concepts and solutions, emphasizes the need for connecting heterogeneous devices within a single global environment, but simultaneously raises a number of questions still remaining unanswered. In order to overcome a number of challenges, such as fragmentation barrier of vertically oriented closed systems, it is crucial that the attention is pointed towards the open platform architecture. Even though there is no universal answer to standardization problem, it should be noted that scalability of applications makes prerequisite for the efficient use of Internet of Things resources.

In this paper we presented an overview of specific IoT related problems and introduced the proposal of an architecture of a microservice based middleware whose aim is to ensure cohesion between different types of devices, services and communication protocols while maintaining the system scalability. In addition to provided concepts, we described a regression model estimation processing dataflow pipeline based on different type data source integration using the proposed architecture.

REFERENCES

[1] Gartner, "Forecast: The Internet of Things", Gartner, 2013.

[2] N. Dhanjani, "Abusing the Internet of Things", O'Reilly Media, 2015.

[3] G. Santucci, "The Internet of Things: Between the Revolution of the Internet and the Metamorphosis of Objects", Vision and Challenges for Realising the Internet of Things, 2010.

[4] M. Abbott and M. Fisher, "The art of scalability: Scalable web architecture, processes, and organizations for the modern enterprise", Addison-Wesley Professional, 2009.

[5] Microservices http://microservices.io/patterns/microservices.html, (20.4.2015.)

[6] M. Richards, "Software Architecture Patterns", O'Reilly Media, 2015.

[7] A. Rodriguez, "RESTful Web services: The basics." https://www.ibm.com/developerworks/webservices/library/ws-restful, (20.4.2015.)

[8] IETF The Constrained Application Protocol (CoAP), http://tools.ietf.org/html/rfc7252

[9] A. McEwen and H. Cassimally, "Designing the Internet of Things", John Wiley & Sons, 2014.

[10] European Research Cluster, "IoT Semantic Interoperability: Research Challenges, Best Practices, Recommendations and Next Steps", 2015.

[11] Nokia Network, LTE-M – "Optimizing LTE for the Internet of Things", 2015.

[12] Wireless M-Bus, "Communication systems for meters and remote reading of meters - Part 4: Wireless meter readout (Radio meter reading for operation in SRD bands)", 2013.

[13] Modicon, "Modicon Modbus Protocol Reference Guide", Modicon Inc. Industrial Automation Systems, 1996.

[14] Modbus.org, "MODBUS Messaging on TCP/IP Implementation Guide V1.0b", 2006.

[15] A. Smola and S.Vishwanathan, "Introduction to Machine Learning", University of Cambridge, 2010.

[16] T. White, "Hadoop: The Definitive Guide", O'Reilly Media, 2009.

[17] P. Chapman, T. Reinartz, C.Sheare and R Wirth, "CRISP-DM 1.0: Step-by-step data mining guide", CRISP-DM consortium, 2000.

[18] L. Schmidt-Thieme, "Machine Learning: Linear Regression", University of Hildesheim, 2007.